

31338 Network Servers

Lab 2c: Manage Processes in Linux and Windows

Aims

1. View information about running processes in both Linux and Windows
2. Start processes with specified priorities in Linux, and change the priority of running processes
3. Get familiar with shell job control in Linux

Task 1: Viewing process information in Linux

Run the command in Linux:

```
ps
```

Which processes does it show by default? Now re-run the command with the *-ef* options. What does it show? Finally run the command with both the *-ef* options and *--forest* (note two dashes for forest). What does it show? It might be helpful if you pipe the *ps* output to *more* (or *less*), so you can see the column headings. Also, note the process ID (PID) of *init*. Recall that *init* is the precursor of all processes in Linux and has a PID of 1 (one).

Now view a continually updated list of running processes using the *top* command. Which process(es) are often near the top of the list? (Just note one or two). The *top* command also shows other useful information. How much physical memory is installed in your virtual machine? How much of the physical memory is currently being used? How much swap space is there, and how much is being used?

If you are in the lab, run the *top* command on your host workstation (Redhat Linux, but not the virtual machine). Record the same kind of information - which process(es) are near the top of the list? how much physical memory and swap space is installed and is being used on the workstation OS?

Task 2: Viewing and changing process priorities in Linux

Now go back to the virtual machine. Run the command:

```
dd if=/dev/zero of=/dev/null
```

This command will continually read input from the "zero" device (source of zeroes) and writes it to the "null" device (often called the "bit bucket" where unwanted output is dumped). The purpose of running this is simply to start a process that will consume a lot of system CPU, so it is possible to monitor it using *top*. Leave the command running in the foreground.

Open a second terminal & run the `top` command. You should see the "dd" process at the top of the list, as it is now consuming the most CPU. Notice in the top listing the column labelled "NI" ("nice" value). What is the default nice value of the dd process?

Make a note of the process ID (PID) of the dd process. Exit from top (press "q") and then use the kill command to kill the dd process by specifying its process ID. Switch back to the other terminal where dd was running. What appears on the screen?

Now re-run the dd command, but with a nice value:

```
nice -n 15 dd if=/dev/zero of=/dev/null
```

Run top in the other console and check the nice value.

Start another terminal and run the command:

```
renice -20 <PID>
```

replacing <PID> with the PID number of your dd process. Check the "nice" value in the console where top is running. What does it show? Does this mean the process has high priority or low priority? Try running some other commands like `ls` or `rpm -qa`. What do you notice?

Now change the process' priority again with:

```
renice 19 <PID>
```

What does this do to the process, and to your system overall when you run other commands in third console window?

Note: The performance of our workstations is high, so you may not notice much differences, so observe carefully! You might also need to start some graphical apps to try and test which priorities make the machine feel slower or faster.

Task 3: Job control in Linux

Go back to the first console window where the dd command should still be running. Suspend the command by pressing `Ctrl+Z`. What message is shown when you press `Ctrl+Z`? Note that the process has not been killed - it is still a running process. Type the command:

```
jobs
```

This shows all jobs (processes) associated with the current shell, and their statuses. Make the dd process continue running in the background by typing:

```
bg 1
```

where "1" is the job number shown as output from the jobs command. Run the jobs command again - what does it say the status of the dd process is this time? When a process is moved to the background it is the same as if you had originally started the process with an ampersand (&) on the end of the command.

Bring the dd command back to the foreground by typing:

```
fg 1
```

You'll notice that anything you type now is just echoed to the screen, and there is no response (at this time) from the shell. When a process is running in the foreground of a shell, it takes control of the input from the keyboard. If the running process does not consume the keystrokes you typed, the shell will receive these when it resumes after the foreground process terminates. Only then will the shell respond.

To conclude, end the `dd` process by pressing `Ctrl+C`. This should terminate the process (the same as if you had used the `kill` command from another console).

Important note: For some strange reason, students sometimes use `Ctrl+Z` to "get out of" an editor. This is **WRONG** and often leads to confusion and errors, as the editor and open files are still "there", just suspended. Please leave your edit sessions by properly terminating the editor, after saving or abandoning, as appropriate, your open files.

Remember – `Ctrl+Z` only suspends a foreground process, but does not kill it. To kill a foreground process, you must use `Ctrl+C`.

Task 4: Viewing processes in Windows

To view processes in Windows using the GUI you should run Task Manager. One way is to right-click in the Windows Taskbar and choose "Task Manager". (The Taskbar is the bar across the bottom of the screen with application icons). The other way is to use the key-combination `Ctrl+Shift+Esc`. When the Task Manager appears, by default it shows only foreground running applications. To show more, choose "More details" near the bottom and you should see a tab showing "Processes".

You can see even more again in the "Details" tab. You can sort by any column by clicking on the column heading, for example to see which processes are using the most CPU or the most memory.

In the Details tab, you can also choose what columns you want displayed - there are in fact many hidden columns with even more information about processes that an administrator can use to diagnose problems on a system. In the details tab, right-click on the column headings and choose *Select columns*. A separate window should appear allowing you to choose other columns you'd like displayed for each process.

Document this in your journal.

You can also see process information from the terminal in Windows using PowerShell. Open a PowerShell window and type:

```
Get-Process
```

Try using the commands to achieve the following tasks in Windows and document the results in your journal. Refer to the documentation page, which has similar examples to these.

1. Use *Get-Process* to show only information about the PowerShell process.
2. Show all Windows processes where the "working set" is size 50 MB (use 50,000,000 bytes) or greater.

Note there is another command you can try too:

```
tasklist
```

Task 5: Killing a process in Windows

Start by opening a Command Prompt (not PowerShell) window. We're going to try killing this process several ways.

Begin by using the GUI, open the Task Manager. Search for the command prompt process - it's friendly name that appears in the Process list might be "Windows Command Processor", but if you go over to the Details tab, you find it named "cmd.exe". Right-click on the process in the list and choose "End task".

Using PowerShell, follow the process in the previous task to find the process that corresponds to your Command Prompt window ("cmd" or "cmd.exe"). Note the number in the "Id" or "PID" column for the command prompt.

To kill the process using it's ID in PowerShell, type:

```
Stop-Process -ID <PID>
```

Note that Windows does also let you kill a process using its name, but that can be dangerous if you have several processes running with the same name. For the reason, using the ID is a more targeted method.